

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

1. Introduction:

This document is the system analysis for the software LogiMind, an AI-powered logistics solution designed for Small and Medium Enterprises (SMEs). The current SME logistics operations rely heavily on manual processing, static routing, and on-the-spot decision-making, resulting in inefficiencies, increased costs, as well as unreliable delivery performance.

LogiMind overcomes these challenges by introducing a hybrid decision-support system that combines route optimization together with AI-driven contextual analysis. By utilising real-time data such as weather conditions, traffic, and fuel trends, the system provides optimized delivery routes, risk alerts, and actionable insights through a centralized dashboard.

The purpose of this document is to showcase the system's requirements, functionality, and overall design approach in supporting smarter, proactive logistics operations for SMEs.

1.1. Purpose:

This document will define the technical scope and key design decisions behind the development of LogiMind. It will outline how the system is structured, how data flows across components, and how the AI-driven decision process operates to support logistics optimization for the SMEs.

So, the key element of system that has been covered are as followed:

1. Architecture:

- a. LogiMind is a web-based, modular system built on a hybrid architecture. It integrates a routing mechanism with an AI reasoning layer, backed by external APIs (weather, fuel, and news). This system is designed for scalability and also flexibility, with different components designed to handle routing, AI processing, and dashboard visualization.

2. Data Flows:

- a. All the data is collected from logistics inputs and external sources, after that they will be structured and processed by the system. The AI model processes these data and produces validated outputs, which will then be displayed to users through the dashboard.

3. Model Process:

- a. The system follows a workflow where input data is processed by the AI model to evaluate conditions, perform trade-off analysis, and to generate actionable recommendations.

4. Role of Reference:

- a. This document serves as a reference for developers and stakeholders to understand the system design, ensuring consistent implementation and testing.

1.2. **Background:**

SME logistics operations usually rely on manual processes such as spreadsheets, basic tools, and driver experience for execution and plan. These methods will limit the efficiency and make it difficult to respond to dynamic factors like traffic conditions, fuel price changes, and unexpected disruptions, causing higher costs and inconsistent delivery performance.

1. *Previous Version:*

- a. Existing logistics systems utilised by SMEs are normally manual systems, with limited automation and integration of real-time data. Route planning is often fixed, and decision making depends heavily on human judgement rather than data-driven insights. Advanced enterprise systems exist in the market but they are too costly and complex most of the time for SMEs to adopt.
2. The updated system introduces a hybrid architecture that separates route finding mechanism from intelligent decision-making. A routing engine generates baseline routes, while an AI reasoning component evaluate these routes using real-time data. Additionally, the system integrates external APIs and includes a centralized dashboard for visualization and monitoring.
3. AI-driven decision support with ability to analyse with context given, dynamic route optimization based on real-time conditions, integration of external data such as fuel prices and weather conditions, automated generation of recommendations, centralised dashboard for monitoring.

1.3. **Target Stakeholder**

Stakeholders	Roles	Expectations
Business Owner	View optimized delivery plans, monitor performance, make decisions.	Expect clear recommendations with easy-to-understand dashboards.
Dispatcher	Monitor real-time delivery conditions.	Route optimization, risk alerts and actionable recommendations.
Development Team	Build the platform and maintain it.	Clear API usage contracts, codebase is modular and well-documented architecture.

System Architecture & Design

1.4. High Level Architecture

1.4.1. Overview:

Type	Details
System	Web-based Decision Dashboard
Architecture	Cloud-based architecture with AI-assisted decision maker

LogiMind is designed as a client-server system with a backend that connects multiple API, analyse the data and visualizes them in the dashboard. The system processes logistics data through a routing engine and enhances decision-making using a GLM-based reasoning layer.

The architecture consists of frontend dashboard, backend API fetcher (fuel price, weather) and AI Reasoning. All services can be deployed independently on cloud server environments such as AWS and Google Cloud.

2.1.2 LLM as Service Layer

In LogiMind, the GLM model is implemented as a dedicated AI reasoning service layer, rather than a simple embedded AI module. It functions as an external decision intelligence engine that processes structured logistics data and returns optimized operational recommendations.

2.1.3 Dependency Diagram

- Before sending a request to the GLM API, the system constructs a fused context object.
- Fuel price trends and cost pressure signals
 - Regional weather conditions and risk classifications
 - Hub-level logistics data
 - Operational signals (disruptions)
- Backend aggregates external data, system builds a strict prompt with schema defined output requirements, request sent to GLM API (ilmu-glm-5.1), model returns structured JSON, backend validates response using schema enforcement, valid output forwarded to dashboard.
- Includes strict validation pipeline, such as JSON response is parsed from raw AI output, schema validation to ensure required fields are not empty, invalid output are rejected or replaced with fallback logics, responses are logged for traceability.

- e.
 - Frontend to Backend API (request recommendation)
 - Backend to External API (weather, fuel price)
 - Backend to GLM API (decision making)
 - Backend to Frontend (update dashboard with new data)
- f. The system follows a distributed interaction flow where the frontend dashboard sends user requests to the backend API. Then the backend coordinates with the routing engine to generate baseline routes, retrieves real-time data from external APIs such as weather and fuel services, and to forward the combined data to the GLM AI service for decision-making. The AI-generated recommendations are then processed and also validated. Finally, the backend returns the optimized results, risk alerts, and insights to the frontend for dashboard display. .
- g. It is an all in one system where the frontend, backend, external APIs fetching is done in one server, except GLM AI system.

2.2.2. Sequence Diagram

- a. Business owners generate the recommendation and use it as a reference.
- b. Business owners open the system, click generate recommendations, wait for the recommendations, view the recommendations, make decisions.
- c. Business owners open the system, click generate recommendations, wait for the recommendations, view the recommendations, make decisions.

1.5. Technological Stack

- 1.5.1. Frontend: Astro and Bun
- 1.5.2. Backend: Astro and Bun

1.6. Key Data Flows

- 1.6.1. The Key Data Flows section describes how data moves through LogiMind from external sources, users, internal processing services, and storage systems. It ensures that all inputs are properly transformed, validated, and stored before being presented as decision outputs on the dashboard.

1.6.1.1. Data Flow Diagram (DFD)

- a. The Data Flow Diagram illustrates the movement of data between external entities (users and APIs), system processes, and the database. Users submit requests through the frontend, which are processed by the backend and routed through services such as the routing engine, data fusion layer, and GLM AI service. External APIs provide real-time inputs like weather and fuel data, which are integrated into the system. The processed output is validated and either stored in the database or returned to the user. The design ensures

proper handling of inputs and avoids issues such as data loss or “black hole” processes where data is not stored or returned.

1.6.2. Normalized Database Schema

- a. There is no database in the project.

2. Functional Requirements & Scope

This section defines the core boundaries of LogiMind and focuses on the Minimum Viable Product (MVP) to demonstrate the system’s technical feasibility. The MVP includes only essential features required to validate AI-driven logistics optimization for SME operations.

2.1. Minimum Viable Product:

2.1.1.

#	Features	Description
1	Data Aggregation & Context Fusion	Collects logistics data (hubs, fleet, deliveries) and external data (weather, fuel prices) to build a unified decision context.
2	AI-Powered Route Optimization	Uses GLM to analyze fused data and generate optimized delivery routes and recommendations.
3	Risk & Disruption Alerts	Detects operational risks such as weather disruptions, fuel price spikes, and congestion, and classifies severity levels.

2.2. Non-Functional Requirements (NFRs)

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g)
Scalability	The system must handle increasing logistics data (hubs, fleets, external signals) without performance degradation.	Modular backend design with independent services for routing, data fusion, and AI processing.
Reliability	The system must ensure consistent AI responses and prevent failure propagation from external APIs.	Fallback mechanisms, schema validation, and error handling for GLM and external API failures.

Maintainability	System components should be easy to update and debug independently.	Separation of logic into modules (routing, AI service, data layer) with clean API interfaces.
-----------------	---	---

2.3. Out of Scope / Future Enhancements

Do identify the high-level features that the team planned for “Grand Vision” but not part of this UMHackathon 2026. [e.g. below]

- a. Driver Mobile Application: A dedicated app for drivers with live navigation, task updates, and communication features.
- b. External System Integrations: Integration with ERP systems, enterprise logistics platforms, or e-commerce platforms.

3. Monitor, Evaluation, Assumptions & Dependencies

3.1. Technical Evaluation:

- 3.1.1. **Grayscale Rollout & A/B Testing:** New features in LogiMind (such as AI recommendation logic or dashboard updates) are deployed gradually to a small percentage of users or test environments first. System performance, response accuracy, and stability are monitored before expanding rollout to all users. A/B testing is used to compare different prompt strategies or model configurations to identify the most effective decision outputs.
- 3.1.2. **Emergency Rollback (ER) & Golden Release:** In case of system anomalies such as invalid AI outputs, API failures, or performance degradation, an emergency rollback mechanism is triggered. The system reverts to a stable “golden release” version of the application, ensuring that core logistics operations and dashboard functionality remain uninterrupted.

3.1.3. Priority Matrix:

3.1.3.1.

Priority	Condition (Trigger)	Action
----------	---------------------	--------

P1 – Critical	GLM API failure, invalid AI outputs affecting all users, or system downtime > 5 minutes	Immediate emergency rollback to last stable release (Golden Version) and disable AI service fallback mode.
P2 – High	Incorrect recommendations, external API failures (weather/fuel), or latency > 5 seconds (p95)	Switch to fallback logic, log error, and trigger monitoring alert for engineering review.
P3 – Medium	Partial data loss (e.g., one region missing weather data) or degraded performance	Continue system operation with degraded mode and retry failed services in background.
P4 – Low	UI glitches, minor formatting issues, or non-critical logging errors	Log issue for future fix without affecting system operation.

3.2. **Monitoring:**

3.2.1. LogiMind includes a continuous monitoring system that tracks key metrics such as API response time, GLM success rate, external API availability, and overall system latency. Health checks are applied to all critical components, and alerts are triggered when thresholds are exceeded, such as high latency or service failure. When issues are detected, the system can switch to fallback logic, log errors for debugging, and notify the development team, ensuring the platform remains operational even under degraded conditions.

3.3. **Assumptions:** State key operational & environmental conditions that you assumed to be valid while development of system and deployment. [e.g.

- a. Users have stable internet connections on their devices during the placement of orders
- b. Restaurants have their own devices to access the web-dashboard for monitoring of orders]

3.4. **External Dependencies:** Do list all the external tools that are applied to your system to function. [e.g. below]

Tools	Purpose	Risks
Z Ai GLM API	It should be the core LLM reasoning engine - that will handle the prompt	High- in case of GLM API rate limitation - core product will fail. So, implement a mitigation strategy [e.g.

	inference, multi-step reasoning and generating structured output	graceful degradation state from users]
Open-Meteo API	Provides real-time weather data for risk and route optimization	API downtime, rate limits, or inaccurate weather data
Fuel Price API	Supplies fuel cost trends for cost-aware logistics decisions	Data inaccuracy, latency, or service unavailability

4. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

- 4.1. **Project Timeline:** Show the timeline of development and justify accordingly [e.g. Day 1-2, Project Planning etc.]
 Day 1-3: Waiting for Masterclass
 Day 4-6: Waiting for API
 Day 7-9: Website development
 Day 10: Documentation
- 4.2. **Team Members Role:** Highlight the role of team members [e.g. Person A – Role Backend Lead – Fast API Microservices & Schema Design]
 Lee Hoon Lim: Documentation
 Lim Jing Heng: Backend development
 Liew Jay Yong: Frontend development
 Lee Xiang Le: Documentation
 Tan Zhi Heng: CI/CD and Documentation
- 4.3. **Recommendations:** If any recommendation is applicable in terms of scalability, performance and reliability [e.g. using Redis Cache for Menu data]
 Use openrouteservice API for route optimization to reduce cost.